

Technical architecture and engineering overview

A detailed reference to the current system: its code structure, technology stack, data flows, security boundaries and engineering decisions.

PREPARED

10 September 2026

SOURCE SNAPSHOT

Main branch · 26ab112

COVERAGE

21 sections · Full-stack architecture

INTENDED READERS

Engineers, operators and product owners

Implementation is not activation.

This document distinguishes built functionality, configuration-dependent availability and external verification. It contains no credentials or customer data and does not certify a production launch.

Contents

1. Product scope and architectural history

3. Technology stack

5. Frontend architecture

7. Backend structure and request lifecycle

9. Video import and processing lifecycle

11. Authentication, ownership and admission

13. Billing and subscription architecture

15. Runtime, deployment and operations

17. Testing and verification strategy

19. Current limitations and sensible extension boundaries

21. Source map and further reading

2. High-level architecture

4. Repository and workspace structure

6. API contract and generated-code workflow

8. Persistence and data model

10. Search, evidence and playback

12. Allowances, quotas and cost controls

14. Security and failure-handling principles

16. Configuration model

18. Key technical decisions and trade-offs

20. Contributor workflow and command reference

Prepared: 10 September 2026

Source snapshot: 26ab112 on the main branch

Audience: engineers, technical collaborators, operators, and product owners

This document describes the current implementation, its structure, and the reasoning behind its major engineering choices. It distinguishes code that exists from features that have been activated or certified against live external services. It is an architecture reference, not a production-readiness certificate.

No credentials, customer records, private object locations, or signed URLs are included. Versions below describe the current technology families and declared dependencies; the lockfiles are authoritative for exact resolved versions.

1. Product scope and architectural history

Scenelt lets a user provide an authorized video, describe a scene or event in natural language, and inspect ranked matches with timestamps and playback.

The project began as a deliberately narrow proof: index one authorized video with Twelve Labs, search it, and compare the indexed source with a separately supplied YouTube reference. It subsequently gained private imports, accounts, source playback, controlled-pilot safeguards, email trials, and subscription infrastructure.

Two product paths remain intentionally distinct:

- **Preserved shared proof:** one fixed demonstration video, shared search history among admitted pilot participants, and its own cumulative search budget.
- **Private imports:** owner-scoped imports, processing state, searches, evidence, and authorized source playback. Private data is not added to the shared demo.

The current application is not a general video library, collaborative sharing service, cross-video search engine, native mobile application, or video editor. It works around individual imports, with a restriction on concurrent active imports per owner.

An important historical decision was to **keep Flask** rather than replace the backend framework. The approved work focused on ingestion correctness and evidence, not a framework rewrite. The React application was built as a new client surface; it should not be described as a migration of an existing Flask/Jinja frontend whose source was never supplied.

2. High-level architecture

The system is best described as a **modular Flask application with a separately operated worker and a React single-page application**. It is not a fleet of independently deployed application microservices.

```

Browser: React + TypeScript + Vite
|
| Same-origin JSON APIs, session cookies, CSRF headers
v
Managed application routing
|
v
Flask API / Gunicorn
|-- PostgreSQL: identity, jobs, results, quotas, billing journals
|-- Private object storage: source media and cached demo stills
|-- Replit OIDC / Firebase Auth: identity verification
|-- Stripe: hosted billing and verified lifecycle events
|
| Durable import records; not an in-memory queue
v
Separate private-import worker
|-- PostgreSQL claims, leases, fencing and heartbeat
|-- Bounded retrieval, ffprobe / ffmpeg, hashing
|-- Twelve Labs upload, indexing and status reconciliation
`-- Cancellation, expiry and resource cleanup

Browser -- scoped upload session --> Private object storage
Browser <-- owner-authorized media -- Flask API
Browser ----- hosted payment flow -----> Stripe

```

Responsibility split

- **Browser:** presentation, file selection, transfer progress, cancellation controls, query caching, playback controls, and authentication/billing navigation.
- **Flask API:** authoritative validation, authentication, authorization, transactional reservations, state changes, search orchestration, media access, and billing operations.
- **Import worker:** browser-independent ingestion, reconciliation, cleanup, and background preparation of preserved-demo stills. Private-import match frames use a different request-time path.
- **PostgreSQL:** durable coordination and business state, including work that may have an uncertain external outcome.
- **Object storage:** persistent private binary media and cached preserved-demo stills. Database rows reference objects and generations; the application does not put video bytes into relational rows. Private-import match JPEGs are generated on request, not saved as durable evidence objects.
- **External services:** Twelve Labs supplies semantic video indexing/search; identity providers verify users; Stripe operates hosted payment interfaces.

There is no application-owned GPU inference service or local semantic vector database. Twelve Labs owns the video index and semantic ranking. There is also no Celery/Redis broker in the current import architecture: durable job coordination uses PostgreSQL.

3. Technology stack

Layer	Technology	Current role
Frontend language	TypeScript 5.9	Typed application, component, and API-client code
UI runtime	React 19	Browser single-page application
Frontend tooling	Vite 7	Development server and production client build
Routing	Wouter 3	Lightweight client-side routing with a configured base path
Server state	TanStack Query 5	Queries, mutations, polling, cache invalidation
Styling	Tailwind CSS 4	Utility styling and semantic design tokens
UI primitives	Radix/shadcn-style components	Dialogs, controls, accessible primitives
UI support	Lucide, CVA, clsx, tailwind-merge	Icons and component class composition
Backend language	Python 3.13+	Application and worker implementation
HTTP application	Flask 3	API routes, authentication and domain services
Production HTTP server	Gunicorn	WSGI process/thread and request-runtime configuration
Validation	Pydantic 2	Strict Python request/domain validation
Relational persistence	PostgreSQL + psycopg 3	SQL-owned schema, transactions, locks and durable records
Provider HTTP	httpx	Twelve Labs adapter and bounded external HTTP work
Video analysis	Twelve Labs API v1.3	Remote indexing, processing and semantic search
Media tools	ffprobe and ffmpeg	Inspection, decoding and evidence-frame extraction
Link resolution	Pinned yt-dlp	Constrained retrieval for permitted non-YouTube sources
Object storage	Google Cloud Storage client / App Storage	Private media, object generations and upload sessions
Pilot identity	Replit OIDC + Authlib	Verified login using OIDC and PKCE

Layer	Technology	Current role
Email identity	Firebase Auth JS 12 + Firebase Admin 7	Email/password, verification and reset, server token verification
Payments	Stripe Python SDK 15 family	Hosted Checkout/Portal and verified billing events
API contract	OpenAPI + Orval + Zod 3	Generated TypeScript client/hooks and validation schemas
Dependency management	pnpm workspaces + uv	JavaScript workspace orchestration and locked Python dependencies
Tests	Python unittest, TypeScript behavior tests, Playwright	Backend, isolated-database, contract and browser checks

Some packages are inherited scaffolding or general UI inventory. A declared dependency is not proof that it is central to the product. In particular, **Express and Drizzle are present in the scaffold but are not the running application backend or the owner of the current SQL schema.**

The Python manifest uses minimum version constraints for most libraries, while `uv.lock` records resolved versions. The link extractor is explicitly pinned, reflecting the sensitivity of extractor behavior to upstream changes.

4. Repository and workspace structure

```
/
|-- artifacts/
|   |-- sceneit/                React product application
|   |   |-- src/pages/          Product, auth, import, demo, billing pages
|   |   |-- src/components/     Upload, search, playback, evidence, UI
|   |   |-- src/lib/            Browser helpers and local behavior
|   |   |-- tests/              Browser cases and synthetic fixtures
|   |   `-- vite.config.ts
|   |
|   |-- api-server/
|   |   |-- sceneit/            Python application and worker modules
|   |   |   |-- migrations/     Versioned SQL migrations
|   |   |   |-- server.py       Flask application entry
|   |   |   |-- import_worker.py Private ingestion/cleanup worker
|   |   |   |-- worker.py       Preserved-proof/operator commands
|   |   |   |-- tests/          Python boundary and persistence tests
|   |   |   |-- docs/           Architecture, operation and audit references
|   |   |   `-- src/            Unused original Node/Express scaffold
|   |   |
|   |   `-- mockup-sandbox/     Isolated design/component preview service
|   |
|-- lib/
|   |-- api-spec/               OpenAPI source and Orval configuration
|   |-- api-client-react/       Generated client/types/hooks and custom fetch
|   |-- api-zod/                Generated Zod schemas
|   |-- replit-auth-web/        Shared browser session/authentication logic
|   `-- db/                     Original Drizzle scaffold; not schema authority
|
|-- scripts/                    Quality, contracts, migration and release tools
|-- pyproject.toml / uv.lock     Python dependency declarations and lock
|-- package.json                Root orchestration commands
|-- pnpm-workspace.yaml          Workspace packages and shared version catalog
|-- pnpm-lock.yaml              JavaScript dependency lock
|-- tsconfig*.json              Shared TypeScript/project-reference settings
`-- replit.md                   Project scope, operating rules and entry points
```

The declared workspace patterns include `artifacts/*`, `lib/*`, `lib/integrations/*`, and `scripts`; the integrations pattern is reserved configuration, not a currently populated package directory. Node/pnpm orchestrates both JavaScript builds and Python commands; a package called `@workspace/api-server` does not imply a Node HTTP implementation.

The mockup sandbox is a design tool, not a second production frontend or a customer-facing part of Sceneit. Its isolated previews should not be confused with the live application.

5. Frontend architecture

Entry, routing and providers

`artifacts/sceneit/src/main.tsx` mounts the React root and application error boundary. `App.tsx` composes routing, the shared authentication provider, TanStack Query, tooltips and notification UI.

The principal client routes are:

Route	Purpose
<code>/</code>	Import entry: MP4/link choice and onboarding context
<code>/auth</code>	Email sign-in/signup and related recovery states
<code>/auth/action</code>	Verification and password-reset action handling
<code>/imports/:id</code>	One private import: status, upload, results and playback
<code>/demo</code>	Preserved shared proof, behind pilot admission
<code>/billing</code>	Billing status, usage and eligible hosted billing actions
Fallback	Not-found state

Wouter uses the configured Vite base URL. The frontend build goes to `dist/public`. Vite requires the managed `PORT` and `BASE_PATH`, binds to all interfaces, and uses a strict port rather than silently selecting an unrelated one.

The Vite configuration does **not** define a local API proxy. Browser API calls use the generated `/api` contract/custom fetch through the application's managed routing. Deployment routing must preserve that contract; changing only the frontend base path does not automatically relocate every API endpoint.

State management

The application separates:

- **Server state:** imports, results, history, session/configuration, billing status and allowances, held in TanStack Query.
- **Ephemeral UI state:** selected files, active scene, query input, modal visibility and transfer progress, held in React state/refs.
- **Authentication coordination:** centralized in `lib/replit-auth-web`, not duplicated across page components.
- **Limited recovery state:** sanitized navigation/import intent and billing idempotency state, used to resume user interaction without automatically repeating expensive actions.

Import and proof pages poll nonterminal processing states. Mutations invalidate relevant detail/history/current-import caches. Shared auth handling and application-level cache resets prevent another account's private state from remaining visible after an identity change.

Cached results are a performance and usability mechanism, not an authorization mechanism. The API rechecks ownership and permissions regardless of what the browser has cached.

Upload controls and cancellation

File bytes are transferred with `XMLHttpRequest`, which provides upload progress and an explicit abort mechanism. An `AbortSignal` connects the page-level cancellation decision to the active transfer.

The implementation guards continuation after reservation, transfer and completion, so a late response cannot silently advance a cancelled browser operation. Component cleanup aborts an active transfer. A failed server-cancellation request leaves honest stopped/retry controls rather than automatically resuming the upload.

An in-app confirmation dialog explains that cancellation schedules private-media cleanup and does not replenish used allowances. Browser transfer abort, server acknowledgement and completed cleanup are separate outcomes.

Visual system and accessibility

The current UI uses a dark slate palette with restrained sage accents, semantic color/radius tokens, Plus Jakarta Sans for interface text, and JetBrains Mono where appropriate. Tailwind utilities and reusable Radix/shadcn-style wrappers provide consistent composition.

Focus-visible and reduced-motion styling exist, but component-library accessibility does not constitute a complete accessibility audit. A known heading/navigation improvement was not completed, and real assistive-technology verification remains separate.

The application is a browser SPA, not an Expo application, native iOS/Android client, or server-rendered Next.js application.

6. API contract and generated-code workflow

The central API description is `lib/api-spec/openapi.yaml`. Orval generates:

1. TypeScript request/response types and API functions.
2. React Query hooks in `lib/api-client-react/src/generated/`.
3. Zod schemas/types in `lib/api-zod/src/generated/`.

The client uses a shared custom fetch implementation for transport, response/error handling and request options. Protected mutations attach the CSRF header supplied by the authenticated session context.

The contract does not generate the Flask application. Python handlers and Pydantic models must implement the declared behavior. Contract tests therefore need to validate actual route emissions, not merely confirm that a hand-written fixture satisfies the generated schema.

Main API families include:

- `/api/auth/session` and the Firebase authentication endpoints.
- `/api/proof` and preserved-proof search/evidence operations.

- `/api/imports` and owner-scoped upload, completion, cancellation, search and media operations.
- `/api/billing/status` , hosted session creation, change preview/confirm/withdraw and webhook handling.
- `/api/healthz` and `/api/readyz` .

Change rule: edit the OpenAPI source, regenerate, update the Python implementation and client usage, then run contract/type checks. Generated files are not hand-edited.

`check:contract-drift` regenerates and compares generated trees, helping detect changes that would otherwise leave backend, types and browser behavior out of sync.

7. Backend structure and request lifecycle

The Flask application factory and module-level application live in `sceneit/server.py` . Domain logic is split into focused Python modules rather than embedded entirely in route handlers.

Module/group	Responsibility
<code>server.py</code> , <code>health.py</code>	Application composition, proof endpoints and health
<code>security.py</code> , <code>auth.py</code> , <code>firebase_auth.py</code>	Host/origin policy, sessions, admission and identity
<code>imports.py</code> , <code>import_search.py</code>	Private import HTTP operations and search orchestration
<code>import_worker.py</code> , <code>worker.py</code>	Background private imports and preserved-proof/operator work
<code>provider.py</code>	Twelve Labs protocol, parsing and bounded search transport
<code>private_storage.py</code> , <code>proof_media.py</code>	Private objects, upload sessions, range reads and evidence
<code>db.py</code> , <code>migrate.py</code> , <code>schema.sql</code>	Database boundaries and migration authority
<code>quota.py</code> , <code>import-limit</code> modules	Transactional reservations and usage enforcement
<code>billing*.py</code> , <code>quota_ops.py</code>	Billing lifecycle, provider adapter, notifications and reconciliation

A protected operation generally follows this order:

1. Enforce host, origin, body-size and route-level security policy.
2. Resolve a verified server session.
3. Check the relevant admission, ownership and/or entitlement rule.
4. Validate the request and enforce current lifecycle state.

5. Acquire bounded capacity and transactional business reservations.
6. Perform the allowed action, preserving external-operation uncertainty.
7. Persist the outcome and return a constrained response.

Not every operation has identical authorization: shared proof admission, private ownership and paid entitlement are deliberately different checks.

Flask startup configures the application; it does not upload media, create provider indexes, or migrate the database. Production serving uses Gunicorn rather than Flask’s development server.

8. Persistence and data model

PostgreSQL is the application authority

The backend uses psycopg with dictionary rows and explicit SQL. It does not use the inherited Drizzle package to manage these tables.

Connection admission is bounded before opening database sockets, with configured connection, statement, lock and idle-transaction timeouts. Application limits must be paired with an appropriately constrained database role; process-local controls alone do not prove a deployment-wide connection ceiling.

Transactions commit when the connection context exits successfully. Worker operations additionally use appropriate autocommit paths, advisory locks and row-level coordination.

Major entity groups

Group	Durable information
Preserved proof	Source/provider identities, metadata, searches and evidence
Private imports	Owner, source metadata, lifecycle, provider identifiers, media generation, expiry and leases
Upload attempts	Attempt identity, reservation/uncertainty state and attributable storage/work usage
Search operations	Normalized query identity, execution state, ranked matches and resolution markers
Authentication	Application owners, provider identities, server sessions and revocation/revalidation state
Trial ledgers	Durable eligibility and consumed cumulative allowances
Capacity/throttles	Bounded resource permits, leases and rate-limit accounting
Commercial quota	Usage windows, reservations, concurrent storage occupancy and audited releases
Billing	Customers/subscriptions, verified coverage, event/operation journals and scheduled changes
Notifications	Outbox state, bounded delivery attempts and ambiguous outcomes

Provider identifiers and object generations are not incidental metadata: they are part of the consistency and cleanup model.

Migrations

Versioned Python-owned SQL migrations and the migration runner define schema evolution. Applied migration history is validated; existing migrations should not be casually rewritten.

Development upgrades use an explicit development migration command or the approved post-merge setup path. Production migration is an explicit operator action, separate from build and web/worker startup.

Database and object storage backups must be coordinated. A database-only restore does not prove that referenced media exists at the required object generations.

Private match-frame URLs are derived views, not references to independently persisted JPEG objects. Their source media and database match metadata matter for recovery; a frame URL by itself is not a backed-up asset.

9. Video import and processing lifecycle

Entry and upload

1. `POST /api/imports` validates the request, canonicalizes any supported link, enforces idempotency/owner rules, and creates durable import state.
2. For an MP4, `POST /api/imports/{id}/upload` validates filename/type/declared size and reserves an upload attempt and applicable allowances before obtaining storage authorization.
3. The browser receives a scoped resumable storage session and sends file bytes directly to object storage.
4. `POST /api/imports/{id}/complete` verifies the actual stored object, including size, content type and immutable generation, before queueing processing.
5. The worker claims eligible work from PostgreSQL and progresses the import independently of the browser.

Storage resumability should not be confused with automatic browser-file restoration. The user may need to reselect a local file after a reload. Durable server processing state can survive browser navigation; an in-memory `File` object cannot.

Source retrieval and inspection

YouTube links are reference/context plus an authorized MP4, not downloader input. Other supported platforms use constrained direct retrieval only when permitted and technically safe; otherwise the application requests a file.

The worker uses bounded retrieval and media inspection. `ffprobe/ffmpeg`, hashes, codecs, dimensions, audio availability and duration checks establish what was actually uploaded. Same-owner fingerprint deduplication can reuse eligible resources; it is not cross-user media sharing.

Provider pipeline

The provider sequence is explicit:

```
Obtain authorized source
-> validate media
-> create provider index
-> upload source asset
-> poll source-asset processing
-> index the asset
-> poll indexed-asset processing
-> validate provider/source metadata
-> mark ready
```

The **uploaded source asset ID and indexed asset ID are different identities**. Search must use the indexed identity. Source/provider duration agreement is a consistency check; it does not prove that a linked YouTube edit shares the same timeline.

State model and recovery

The lifecycle vocabulary includes `awaiting_upload`, `file_required`, `queued`, `resolving`, `validating`, `uploading`, `processing`, `indexing`, `ready`, `cancel_requested`, `cancelled`, `expired`, `failed` and `needs_review`.

These are not a single strictly linear chain: fallback, cancellation, expiry and uncertain external writes create branches. In particular, a browser upload reservation remains in an awaiting-file/upload state; provider `uploading` is a different processing stage.

The worker uses row claims, leases, heartbeats and fencing checks. A worker that loses ownership must stop making new provider changes or committing stale outcomes. External-write intent is recorded before sending a mutation; an ambiguous restart does not silently repeat the purchase/work.

Cancellation and retention

The cancel endpoint records `cancel_requested`. The worker revokes upload sessions, reconciles provider-side work and deletes resources in dependency-aware order, while retaining resources still referenced by another valid same-owner import.

Access expiry and physical deletion are different guarantees. Access can be closed while uncertain provider cleanup still requires operator review. Similarly, the application's upload acceptance deadline does not revoke the underlying storage bearer session: explicit revocation and generation constraints remain necessary.

10. Search, evidence and playback

Private search requires an owned, ready, non-expired import and appropriate allowance/capacity. Audio-only requests also require an audio-capable source.

The search path normalizes query identity and reuses eligible completed identical results. Opening saved results does not make another semantic-search request or consume a new search allowance.

For a new search, the application records a reservation before calling the provider. A guarded child process bounds external search execution and output. A timeout or lost response can become review-required work, not an automatic resubmission.

Returned candidates are checked for:

- The expected indexed-video identity.
- Finite, ordered, nonnegative timestamps inside the source duration.
- Supported confidence values and bounded result counts.
- Partial/discarded-result handling rather than fabricated matches.

The private path returns up to five validated matches. This is application filtering of provider-ranked results, not a locally implemented semantic ranking model.

Evidence paths are not identical

Preserved-demo stills are cached and worker-prepared; their HTTP route does not freely spawn extraction. Private-import frame requests instead perform bounded request-time extraction with resource/quota reservations and return an in-memory, no-store JPEG. Those private JPEGs are not persisted as evidence objects. Both paths need private media handling, but they should not be described as one interchangeable implementation.

Source playback uses the native browser video element and an owner-authorized media endpoint. Range requests, generation checks and bounded streaming support seeking without exposing a public MP4 URL.

Segment controls seek/restart/loop against the indexed source. External-platform links remain references unless actual playback and alignment have been verified. A mobile Chromium viewport, ready player, matching title or matching duration does not certify physical-device playback or linked-edit alignment.

11. Authentication, ownership and admission

There are three separate questions:

1. **Identity:** who is the authenticated application user?
2. **Admission/ownership:** may that user access this product path or resource?
3. **Entitlement:** may that user perform new work under the applicable allowance or paid coverage?

Conflating them would create privacy and billing vulnerabilities.

Replit pilot authentication

Replit OIDC with PKCE verifies identity and establishes an application session. The shared pilot is additionally controlled by an exact, case-sensitive OIDC-subject allowlist. An empty allowlist admits nobody; a valid login alone is not pilot admission.

Firestore email authentication

Firestore Email/Password is additive, not a replacement for Replit login or a migration to Firestore. Firestore handles passwords and its identity flows; the Flask application verifies identity and creates its own secure server session.

The browser uses in-memory Firestore persistence. A short-lived, same-origin challenge and CSRF-bound exchange precede server validation of the ID token and user state. Verification includes the expected project/issuer/audience, validity/revocation, suitable sign-in provider, freshness and required verified-email state.

Firestore identities map to opaque application owners. Email addresses or provider UIDs are not accepted as browser-supplied ownership authority, and matching email text does not automatically merge accounts.

Sessions and recovery

The application session is maintained through secure HttpOnly cookies and PostgreSQL-backed session state. Provider-state revalidation is bounded; uncertain identity status fails closed for protected new/private work.

Protected writes enforce same-origin and CSRF requirements. Return destinations are constrained to safe application paths. Verification/reset parameters are handled through the supported action flow rather than retained in ordinary navigation state.

Logout and account changes coordinate provider state, server session and browser caches. Cross-tab signalling and local access suppression avoid leaving private content available while signout/recovery status is uncertain.

Current account-mode boundary

Verified email users can have private trial access without becoming shared-proof pilot members. Current initial purchase/upgrade eligibility remains restricted to admitted Replit identities. The billing page exists, but a normal Firestore trial-to-paid conversion is not yet implemented.

12. Allowances, quotas and cost controls

The system has distinct accounting models rather than one universal counter.

Model	Semantics
Shared proof	Independent cumulative proof-search budget
Private pilot/trial	Cumulative/lifetime import and search accounting
Commercial plans	Finite usage windows backed by verified paid coverage
Storage	Concurrent occupancy, not a monthly consumptive counter
Runtime capacity	Concurrency/rate/lease controls, separate from purchased entitlement

Documented baseline private limits include 200 MB media, a 4–1,200 second duration range, seven-day access/retention policy, and lifetime allowances of three import attempts and fifty new searches per account/trial ledger. Baseline application-wide limits are separate. These are not approved commercial tier prices or allowances; commercial limits come from reviewed configuration.

Trial eligibility uses a stable keyed digest of conservatively normalized verified email, separate from resource ownership. Recreating an identity must not automatically replenish an already-consumed trial. Changing the eligibility hash secret is therefore an accounting migration, not routine credential rotation.

Reservations are transactional and idempotent. Failed, cancelled or uncertain work can remain consumed. Deleting an import, expiring media or signing out does not refund a lifetime allowance.

Commercial monthly windows are anchored to confirmed coverage. Yearly billing does not mean a year's worth of monthly allowance is made available at once. Verified upgrades may raise a ceiling without resetting already-used capacity or changing the usage anchor.

Release of unused work/storage reservations requires durable evidence. A timeout, process exit or lease expiry alone does not prove that a provider did no work or that an object was deleted.

These controls bound application-approved work. They are not a universal guarantee against every possible infrastructure egress, storage or third-party charge.

13. Billing and subscription architecture

Hosted payment boundary

Card numbers and CVC are not handled by Scenelt. The server creates allowlisted Stripe-hosted Checkout or Customer Portal sessions, and the browser navigates to Stripe.

The browser supplies a constrained offer selection and operation identity. The server derives authoritative customer, Price and return-destination details from reviewed configuration; the browser cannot choose an arbitrary payable amount or Price as an entitlement shortcut.

A successful browser redirect is **not** proof of payment. Entitlement comes from verified provider state, processed through signed webhook handling or bounded reconciliation.

Internal separation

Billing modules separate HTTP routes, configuration/catalog validation, provider adaptation, business lifecycle, quota updates, operator recovery and optional notifications.

The React `/billing` page is wired into the router and displays status/usage, eligible offers, hosted management actions, preview/confirmation, scheduled-change withdrawal and payment/recovery states. Its disabled or ineligible presentation is deliberate—not evidence that the page is absent.

Lifecycle rules

- Checkout remains pending or uncertain until authoritative payment evidence is available.
- Period-end cancellation preserves already-confirmed paid coverage until its end.
- Failed or unfunded renewals do not mint new coverage or allowance.

- Upgrades use an expiring server-side preview and verified incremental funding.
- Downgrades and billing-cadence changes are scheduled according to renewal rules.
- Refunds/disputes/reversals affect the coverage attributable to the relevant funding, without replenishing consumed work.
- Idempotency, durable operation journals and uncertainty states prevent unsafe repeated actions.

Webhooks and reconciliation

Webhook processing validates the original request bytes, signature/timestamp and environment, deduplicates event identities, and verifies customer/subscription/allowlisted-price relationships.

Event arrival order is not treated as authoritative lifecycle order. Reconciliation resolves relevant provider state, and acknowledgement must correspond to completed handling or a durable recovery record.

Status, account management and cleanup are distinct from permission to buy new work. For example, loss of purchase eligibility should not unnecessarily remove the ability to manage an already-associated billing account.

Notifications

An optional outbox supports bounded billing/dunning/incident notifications. Stripe remains responsible for card retry behavior and its hosted receipts. SMTP transport acceptance is not proof of inbox delivery.

Ambiguous post-send outcomes are not blindly resent. Independent monitoring is still required: an application-hosted outbox cannot reliably report that its own runtime, webhook endpoint or database is completely unavailable.

Activation status

Billing is disabled by default and live mode needs explicit approval. Public trials have a separate default-off switch. Neither switch implies the other, and neither should silently change pilot admission.

The merged Stripe sandbox record documents deferred hosted verification. Local test fixtures and added credentials do not certify Checkout, Portal, signed delivery, renewals, refunds or real receipts. Final selling prices and commercial policy must come from an approved catalog, not test fixtures.

14. Security and failure-handling principles

Boundary	Protection and implication
Private resources	Server-side owner scoping on status, search, evidence and media
Shared demo	Separate exact pilot admission; no implicit admission through email signup
Browser mutations	Same-origin and CSRF checks; UI disablement is not authorization

Boundary	Protection and implication
Retrieval	Constrained hosts/redirects/DNS/media; no arbitrary internal-network fetcher
Storage writes	Exact-size reservations, create-only generation rules and encrypted session references
Storage reads	Generation-pinned private access, range validation and bounded streaming
Provider mutations	Persist intent before sending; preserve ambiguity rather than auto-retry
Worker ownership	Claims, heartbeat, leases, advisory locks and fencing
Expensive execution	Finite permits, deadlines, child-process termination/reaping
Billing	Hosted card handling, verified payment evidence, server-owned catalog
Preview files	Explicit Vite filesystem limits; API admission must not be bypassed by file serving
Logs	Safe identifiers/codes/durations; no tokens, source URLs, queries or media payloads

One governing rule is that **an accounting deadline is not proof of external termination**. A timed-out HTTP call may already have created a provider resource. An expired upload acceptance window may leave a storage bearer usable until revoked. A worker killed during restart may leave a capacity permit until normal expiry.

Consequently, `needs_review` and retained reservations are intentional safety outcomes. Operators reconcile evidence rather than forcing a green status by deleting permits or replaying ambiguous external writes.

Storage-session encryption depends on application key material. Key rotation must account for in-flight encrypted references; it should not make cleanup references unreadable without an explicit recovery plan.

These are implemented controls and design intentions, not a claim of independent penetration testing or complete security certification.

15. Runtime, deployment and operations

Development

The product has separate managed frontend, Flask API and private-import-worker workflows. The mockup sandbox is independently operated for design work. One-off type/test commands are checks, not additional production services.

Dated workspace observation, not a deployment guarantee: managed workflow status inspected on 10 September 2026 at approximately 13:14 UTC reported the web, API and private-import-worker workflows running. This observation comes from workspace status/logs, not from source-code inspection, and is not a certification of production health or successful video processing.

Production topology

Durable ingestion needs a persistent worker; an autoscaled web-only process is not sufficient. A supervised `start:with-worker` option exists, but the operating guidance does not treat it as already activated production infrastructure.

The current worker uses restrictive capacity/advisory-lock ownership. Increasing worker count requires a deliberate coordination design and capacity review, not simply more replicas.

Production serving also needs approved host/proxy settings, private object storage, an appropriately constrained database role, explicit schema migration, backups, and operational monitoring. This document does not inspect or certify a published deployment.

Health and observability

- `/api/healthz` is a cheap liveness signal.
- `/api/readyz` validates application configuration and bounded database/migration readiness.
- Provider readiness is explicitly not probed as a side effect of these endpoints.
- Worker heartbeat/capacity and review-required operations need separate operational attention.
- Logs emphasize request/operation identifiers, route templates, safe status/error codes and duration, rather than private payloads.

This separation prevents health probes from buying analysis work or falsely asserting that media playback and billing integrations are certified.

Migration, backup and release discipline

A release should coordinate paused/new work, database and object backups, disposable restore verification, migration readiness, restart checks and explicit approval.

Production schema upgrades are operator actions, not startup hooks. Development post-merge automation may run the explicit development upgrade path, but that is not permission to migrate production.

Recovery must preserve ownership, generation references, paid coverage and consumed usage. Restoring only convenient subsets can create orphaned media, broken evidence or incorrect entitlement.

16. Configuration model

Configuration has independent identity, access, provider, quota and runtime domains.

Domain	Representative settings	Rule
Core persistence/session	<code>DATABASE_URL</code> , <code>SESSION_SECRET</code>	Private runtime credentials; never client-bundled
Pilot admission	<code>PILOT_ALLOWED_SUBJECTS</code>	Exact identities; absence fails closed
Proxy/host trust	<code>TRUSTED_HOSTS</code> , proxy-hop/forwarding settings	Match the real managed route, not a permissive guess

Domain	Representative settings	Rule
Public trials	<code>SCENEIT_PUBLIC_TRIAL_ENABLED</code>	Separate, explicit rollout switch
Firebase	Public web configuration plus backend-only Admin/trial-hash credentials	Invalid configuration closes email processing; do not expose private credentials
Video provider	<code>TWELVE_LABS_API_KEY</code>	Required for approved provider work, not health probes
Private storage	Approved bucket/object-root configuration	No public MP4 fallback
Billing	<code>SCENEIT_BILLING_ENABLED</code> , reviewed catalog and Stripe credentials	Validate environment/catalog/approval together
Capacity	Connection, search, media, worker and lease limits	Finite safeguards, not values to raise to hide errors
Notifications	Approved sender/transport/scheduling and enablement	Independent from payment and signup activation

Setting a credential is not the same as approving activation. Enabling billing is not the same as opening email trials, granting shared-proof admission or certifying live payment behavior.

The authoritative exact setting names/defaults are in the relevant configuration modules and operating documents. This table intentionally contains no secret values or private deployment addresses.

17. Testing and verification strategy

Backend and persistence

Python unittest covers validation, ownership/session/CSRF boundaries, provider adapters, state transitions, deadlines, worker shutdown, cancellation, billing safety and recovery.

Database-sensitive suites use a separate disposable `sceneit_test*` database and refuse to use the application database as destructive test storage. Tests cover locking, concurrent reservations, migration drift, restart persistence, idempotency, coverage and quota accounting.

The ordinary run may skip these cases when no suitable isolated database is supplied. A green run with skips is not evidence that those database scenarios ran.

Frontend and browser

TypeScript behavior tests cover local state/utility contracts. Playwright exercises signup, recovery, imports, billing, pilot boundaries and complete-journey scenarios using intercepted APIs and synthetic media.

The configured mobile project is an iPhone-sized Chromium fixture, not physical iOS Safari. Browser-level mocks are useful for deterministic states, but must not be used as evidence of real email delivery, actual Stripe hosted behavior or live Twelve Labs processing.

The project also validates generated contracts, build/type integrity and preview filesystem boundaries.

Evidence at this snapshot

Managed workflow output inspected on 10 September 2026 at approximately 13:14 UTC reported the following. This is a dated workspace observation, separate from the older, scoped runs in the checked-in audit documents:

- Workspace type checks passed.
- Python suite: **337 total, 255 passed, 82 skipped**.
- Separate checked-in customer-journey, billing and isolated-database reports provide additional scoped evidence.

The sources were the `typecheck` and `python-boundaries` managed-workflow outputs; the latter reported `Ran 337 tests and 0K (skipped=82)`. These were existing recorded results, not newly triggered paid/provider tests. Counts are a snapshot and should not be treated as a permanent coverage metric.

Earlier real video ingestion/search proof exists, but later fixture-based hardening does not retroactively certify every new live import, email, payment or phone flow.

What still requires external verification

Real phones/Safari and linked-video playback/alignment, approved Firebase project/email delivery, genuine hosted Stripe lifecycles, production runtime/restore configuration, and independent accessibility/security review remain separate from local test evidence.

18. Key technical decisions and trade-offs

Decision	Why it was made	Consequence/trade-off
Keep Flask	Focus on product/ingestion risk rather than an unrelated rewrite	Python remains the authoritative backend despite Node scaffolding
New narrow React client	Provide a maintainable interactive UI without claiming an unavailable legacy UI migration	SPA routing/build and backend contracts must stay coordinated
Separate shared proof/private imports	Preserve original evidence/history while protecting new private data	Two domain paths and admission models must remain explicit
PostgreSQL-backed coordination	Make reservations and job state transactional and durable	More explicit SQL/locking logic; no plug-and-play external queue

Decision	Why it was made	Consequence/trade-off
Separate worker	Continue processing after browser disconnect and keep ingestion out of web requests	Always-on runtime and deployment approval are required
Direct-to-storage upload	Avoid moving large upload bodies through Flask memory	Bearer-session revocation and generation integrity become critical
Private API-mediated playback	Enforce ownership and generation-aware media access	Range streaming and egress accounting add backend complexity
External semantic engine	Use Twelve Labs rather than build video embeddings/ranking infrastructure	Provider availability, pricing and ambiguous operations need controls
No blind external-write retry	Avoid duplicate uploads, searches or charges	Some work needs operator review; quota may remain consumed
Explicit schema migration	Avoid startup races and accidental production DDL	Deployment requires an additional controlled step
Generated API client	Reduce handwritten request/type drift	Generation must be paired with real route-contract tests
Separate identity/admission/entitlement	Prevent login from implicitly granting data or paid access	More explicit eligibility states, including a current trial-to-paid gap
Default-off email/billing rollout	Avoid unapproved public access and charging	Merged code is not a live launch
Hosted card handling	Keep sensitive payment entry at Stripe	Provider-hosted flows still require genuine integration certification
Provider-free quality gates	Repeatable tests without spending user allowances	External delivery, playback and payment proof remain separate

19. Current limitations and sensible extension boundaries

The implementation is substantially beyond the original one-video proof, but several boundaries remain important:

- **Email trial-to-paid conversion:** initial purchase/upgrade eligibility does not yet support ordinary Firebase trial customers. A safe implementation must preserve owner identity and consumed allowances without implicitly granting shared-pilot admission.
- **Activation and certification:** public trials and billing are controlled rollouts; live email and hosted payment evidence is incomplete/deferred.
- **Production worker topology:** current operating guidance requires an approved persistent worker arrangement, rather than assuming the web deployment handles ingestion.
- **Playback evidence:** physical devices and separately linked edits need actual playback/seek checks.

- **Accessibility and analytics:** known navigation/heading work and signup-funnel instrumentation are not complete.
- **Operational burden:** uncertain external work, object generations, coverage journals and coordinated backups require meaningful runbooks and monitoring.
- **Scaffold debt:** unused Express/Drizzle and broad UI inventory can confuse new contributors. Their presence must not trigger accidental adoption or database replacement.
- **Scaling:** higher throughput requires measured changes to worker coordination, database limits, media streaming and provider budgets. Raising replicas/limits blindly can invalidate the existing safety model.

No current source is a basis for claiming cross-video search, public sharing, unlimited analysis, native mobile support, production payment certification or automatic trial conversion.

20. Contributor workflow and command reference

Start with `replit.md` and the relevant operation guide. Make the smallest domain-specific change and preserve the current framework, ownership and provider-safety boundaries.

Command	Purpose
<code>pnpm run typecheck</code>	Build shared TypeScript declarations and check workspace code
<code>pnpm run build</code>	Type checks followed by package builds
<code>pnpm --filter @workspace/api-spec run codegen</code>	Regenerate clients and schemas from OpenAPI
<code>pnpm run check:contract-drift</code>	Check generated outputs against the contract
<code>pnpm --filter @workspace/sceneit run test:behavior</code>	Frontend behavior tests
<code>pnpm --filter @workspace/sceneit run test:ui</code>	Playwright fixture/browser tests
<code>pnpm --filter @workspace/api-server test</code>	Python unittest discovery through the locked environment
<code>pnpm run quality:local</code>	Isolated local quality gate with disposable PostgreSQL and controlled fixtures
<code>pnpm run release:check</code>	Locked release gate with its documented isolated-test prerequisites
<code>pnpm --filter @workspace/api-server run dev</code>	Flask development command; use the managed workflow
<code>pnpm --filter @workspace/sceneit run dev</code>	Vite development command; use the managed workflow

Command	Purpose
<code>pnpm --filter @workspace/api-server run worker</code>	Private import worker command; not a web-request action

For schema changes, use the explicit development status/upgrade commands in `replit.md` and add the appropriate SQL migration. Do not run the unused Drizzle push against Python-owned tables.

For API changes, update OpenAPI, regenerate, update handlers/consumers, then validate actual responses and affected flows. For auth, ownership, provider-write or billing changes, include adversarial and uncertain-outcome cases, not only a successful fixture.

When reviewing an unfamiliar path, trace **identity** → **authorization** → **reservation** → **external work** → **durable outcome** → **UI recovery**. That sequence exposes most of this project's consequential failure boundaries.

21. Source map and further reading

All paths below are relative to the workspace root.

Topic	Primary references
Scope and operating rules	<code>replit.md</code>
Dependencies/workspace	<code>pyproject.toml</code> , <code>uv.lock</code> , <code>pnpm-workspace.yaml</code> , <code>pnpm-lock.yaml</code> , package manifests
Frontend entry/routes	<code>artifacts/sceneit/src/main.tsx</code> , <code>artifacts/sceneit/src/App.tsx</code>
Frontend visual/build setup	<code>artifacts/sceneit/src/index.css</code> , <code>artifacts/sceneit/vite.config.ts</code>
Browser auth	<code>lib/replit-auth-web/src/use-auth.ts</code> , <code>lib/replit-auth-web/src/firebase-client.ts</code>
API source/generation	<code>lib/api-spec/openapi.yaml</code> , <code>lib/api-spec/orval.config.ts</code> , <code>lib/api-client-react/src/custom-fetch.ts</code>
Flask/domain boundaries	<code>artifacts/api-server/sceneit/server.py</code> , <code>security.py</code> , <code>auth.py</code> , <code>firebase_auth.py</code>
Imports/search/provider	<code>artifacts/api-server/sceneit/imports.py</code> , <code>import_worker.py</code> , <code>import_search.py</code> , <code>provider.py</code>
Media/persistence	<code>artifacts/api-server/sceneit/private_storage.py</code> , <code>proof_media.py</code> , <code>db.py</code> , <code>migrate.py</code> , <code>schema.sql</code>
Billing/quotas	<code>artifacts/api-server/sceneit/billing.py</code> , <code>billing_routes.py</code> , <code>billing_provider.py</code> , <code>billing_config.py</code> , <code>quota.py</code>
Import operations	<code>artifacts/api-server/docs/import-operations.md</code>

Topic	Primary references
Pilot/release evidence	<code>artifacts/api-server/docs/pilot-operations.md</code> , <code>pilot-verification.md</code>
Email/trials	<code>artifacts/api-server/docs/email-trial-operations.md</code>
Billing operation/activation	<code>artifacts/api-server/docs/billing-operations.md</code> , <code>billing-activation-review.md</code>
Billing delivery/evidence	<code>artifacts/api-server/docs/billing-notification-operations.md</code> , <code>billing-local-verification.md</code> , <code>billing-sandbox-verification.md</code>
Customer journey	<code>artifacts/api-server/docs/customer-journey-audit.md</code> and its evidence directory
Test/release tools	<code>artifacts/sceneit/playwright.config.ts</code> , <code>scripts/release-check.sh</code> , <code>scripts/pilot-local-gate.sh</code> , <code>scripts/validate-openapi-response.mjs</code>

When a historical audit differs from newer source, use the current implementation and update the interpretation. For example, the older audit's absence of a billing UI no longer describes the routed billing page; its warning that email trials do not automatically become paid memberships still applies.

Maintenance note: update this reference when identity/entitlement rules, API generation, schema ownership, worker topology or external-operation recovery semantics change. Do not turn temporary workflow state or a successful mock into a durable claim of production certification.